

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CIS Invoice Template</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <link href="https://fonts.googleapis.com/css2?
family=SF+Pro+Display:wght@300;400;500;600&family=SF+Pro+Text:wght@400;500;600&
display=swap" rel="stylesheet">
  <style>
    body {
      font-family: 'SF Pro Text', -apple-system, BlinkMacSystemFont, sans-serif;
      background-color: #f5f7fa;
      color: #1d1d1f;
    }
    .invoice-container {
      max-width: 900px;
      margin: 2rem auto;
      background-color: white;
      box-shadow: 0 2px 20px rgba(0, 0, 0, 0.08);
      border-radius: 12px;
      overflow: hidden;
      transform: scale(0.95);
      transform-origin: top center;
    }
    .header {
      background-color: #0071e3;
      padding: 1.5rem 2rem;
      color: white;
    }
    .input-field {
      border: none;
      border-radius: 8px;
      padding: 0.5rem;
      width: 100%;
      background-color: rgba(0, 0, 0, 0.02);
      transition: all 0.2s ease;
    }
    .input-field:focus {
```

```
    outline: none;
    background-color: rgba(0, 0, 0, 0.04);
}
.address-box {
    background-color: rgba(0, 0, 0, 0.02);
    border-radius: 8px;
    padding: 1rem;
    transition: all 0.2s ease;
}
.address-box:hover {
    background-color: rgba(0, 0, 0, 0.04);
}
.tab {
    cursor: pointer;
    padding: 0.5rem 1rem;
    border-bottom: 2px solid transparent;
    transition: all 0.2s ease;
}
.tab.active {
    border-bottom: 2px solid #0071e3;
    font-weight: 500;
    color: #0071e3;
}
.table-row:nth-child(even) {
    background-color: rgba(0, 0, 0, 0.01);
}
.divider {
    height: 1px;
    background-color: rgba(0, 0, 0, 0.06);
    margin: 1.5rem 0;
}
.btn {
    border-radius: 20px;
    font-weight: 500;
    transition: all 0.2s ease;
}
.table-input {
    background-color: transparent;
    border: none;
    width: 100%;
    padding: 0.5rem;
```

```
}  
.table-input:focus {  
  outline: none;  
  background-color: rgba(0, 0, 0, 0.03);  
  border-radius: 6px;  
}  
.action-link {  
  color: #0071e3;  
  font-weight: 500;  
  transition: all 0.2s ease;  
}  
.action-link:hover {  
  opacity: 0.8;  
}  
.delete-row {  
  opacity: 0.5;  
  transition: all 0.2s ease;  
  font-size: 18px;  
  width: 24px;  
  height: 24px;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  border-radius: 50%;  
}  
.delete-row:hover {  
  opacity: 1;  
  background-color: rgba(0, 0, 0, 0.05);  
}  
.fade-in {  
  animation: fadeIn 0.3s ease-in-out;  
}  
@keyframes fadeIn {  
  from { opacity: 0; }  
  to { opacity: 1; }  
}  
.empty-state {  
  padding: 2rem;  
  text-align: center;  
  color: #86868b;  
}
```

```

table {
  table-layout: fixed;
}
th:nth-child(1), td:nth-child(1) { width: 12%; }
th:nth-child(2), td:nth-child(2) { width: 15%; }
th:nth-child(3), td:nth-child(3) { width: 35%; }
th:nth-child(4), td:nth-child(4) { width: 10%; }
th:nth-child(5), td:nth-child(5) { width: 12%; }
th:nth-child(6), td:nth-child(6) { width: 12%; }
th:nth-child(7), td:nth-child(7) { width: 4%; }
.fiscal-week {
  display: inline-block;
  background-color: rgba(0, 113, 227, 0.1);
  color: #0071e3;
  border-radius: 12px;
  padding: 0.2rem 0.6rem;
  font-size: 12px;
  font-weight: 500;
  margin-left: 0.5rem;
}
</style>
</head>
<body>
<div class="invoice-container">
  <!-- HEADER SECTION -->
  <div class="header flex justify-between items-start">
    <div>
      <h1 class="text-3xl font-semibold tracking-wide">INVOICE</h1>
      <p class="text-sm mt-1 opacity-90">Construction Industry Scheme (CIS)</p>
    </div>
    <div class="text-right">
      <p class="text-sm opacity-90">Invoice #:</p>
      <input type="text" id="invoice-number" class="input-field bg-transparent text-white text-right" placeholder="" readonly>
    </div>
  </div>

  <!-- FROM / BILL TO SECTION -->
  <div class="p-6 grid grid-cols-1 md:grid-cols-2 gap-6">
    <div class="from-section">
      <h2 class="text-sm font-medium text-gray-500 mb-2">From</h2>

```



```
</div>

<!-- INVOICE ITEMS SECTION -->
<div class="px-6 mt-6">
  <div class="divider"></div>
  <h2 class="text-lg font-medium mb-3">Invoice Items</h2>

  <!-- Tabs -->
  <div class="flex overflow-x-auto border-b border-gray-100 mb-4">
    <div class="tab active" data-day="all">All Days</div>
    <div class="tab" data-day="monday" data-date="2025-07-14">Monday</div>
    <div class="tab" data-day="tuesday" data-date="2025-07-15">Tuesday</div>
    <div class="tab" data-day="wednesday" data-date="2025-07-16">Wednesday</
div>
    <div class="tab" data-day="thursday" data-date="2025-07-17">Thursday</div>
    <div class="tab" data-day="friday" data-date="2025-07-18">Friday</div>
  </div>

  <!-- Table -->
  <div class="overflow-x-auto">
    <table class="w-full border-collapse">
      <thead>
        <tr class="text-left text-sm text-gray-500">
          <th class="py-2 px-3 border-b border-gray-100">Date</th>
          <th class="py-2 px-3 border-b border-gray-100">Location/Plot</th>
          <th class="py-2 px-3 border-b border-gray-100">Description of Works</th>
          <th class="py-2 px-3 border-b border-gray-100 text-right">Hours</th>
          <th class="py-2 px-3 border-b border-gray-100 text-right">Rate</th>
          <th class="py-2 px-3 border-b border-gray-100 text-right">Amount</th>
          <th class="py-2 px-3 border-b border-gray-100 w-10"></th>
        </tr>
      </thead>
      <tbody id="invoice-items">
        <tr class="table-row" data-date="14/07/25">
          <td class="py-2 px-3 border-b border-gray-50">
            <input type="text" class="table-input text-sm date-field"
value="14/07/25">
          </td>
          <td class="py-2 px-3 border-b border-gray-50">
            <input type="text" class="table-input text-sm" placeholder="Location/Plot
Number">

```

```

        </td>
        <td class="py-2 px-3 border-b border-gray-50">
            <input type="text" class="table-input text-sm" placeholder="Description
of work performed">
        </td>
        <td class="py-2 px-3 border-b border-gray-50">
            <input type="number" class="table-input text-sm text-right hours-input"
value="" min="0" step="0.5" placeholder="">
        </td>
        <td class="py-2 px-3 border-b border-gray-50">
            <input type="text" class="table-input text-sm text-right rate-input"
value="" placeholder="£0.00">
        </td>
        <td class="py-2 px-3 border-b border-gray-50">
            <input type="text" class="table-input text-sm text-right amount-field"
value="£0.00" readonly>
        </td>
        <td class="py-2 px-3 border-b border-gray-50 text-center">
            <button class="text-gray-400 delete-row">x</button>
        </td>
    </tr>
</tbody>
</table>
<div id="empty-state" class="empty-state hidden">
    <p>No items for this day yet.</p>
    <p class="text-sm mt-2">Click "Add Line Item" to add one.</p>
</div>
</div>

<div class="flex mt-4 gap-4">
    <button id="add-line" class="action-link text-sm">+ Add Line Item</button>
    <button id="add-weekdays" class="text-green-500 text-sm font-medium">+ Add
All Weekdays</button>
</div>
</div>

<!-- TOTALS SECTION -->
<div class="px-6 mt-6">
    <div class="divider"></div>
    <div class="ml-auto w-full md:w-64">
        <div class="flex justify-between mb-2 text-gray-600">

```

```

        <span>Subtotal:</span>
        <span id="subtotal">£0.00</span>
    </div>
    <div class="flex justify-between mb-2 text-red-500">
        <span>CIS Deduction (20%):</span>
        <span id="cis-deduction">£0.00</span>
    </div>
    <div class="flex justify-between font-medium text-lg">
        <span>Total Due:</span>
        <span id="total-due">£0.00</span>
    </div>
</div>
</div>

<!-- ADDITIONAL NOTES SECTION -->
<div class="px-6 mt-6">
    <div class="divider"></div>
    <h2 class="text-sm font-medium text-gray-500 mb-2">Additional Notes</h2>
    <textarea class="input-field w-full h-24 resize-none text-gray-600"
placeholder="Payment terms, bank details, or any additional notes..."></textarea>
</div>

<!-- FOOTER BUTTON -->
<div class="p-6 flex justify-center">
    <button id="save-invoice" class="bg-blue-500 text-white font-medium py-3 px-8
rounded-full hover:bg-blue-600 transition-colors btn shadow-sm">
        Save Invoice
    </button>
</div>
</div>

<script>
document.addEventListener('DOMContentLoaded', function() {
    // Store the current active day
    let activeDay = 'all';
    let activeDateStr = '';

    // Initialize date mapping
    const dayMapping = {
        'monday': '2025-07-14',
        'tuesday': '2025-07-15',

```



```

    'wednesday': '2025-07-16',
    'thursday': '2025-07-17',
    'friday': '2025-07-18'
  };

  // Calculate fiscal week and set invoice number
  updateFiscalWeekAndInvoiceNumber();

  // Tab switching
  const tabs = document.querySelectorAll('.tab');
  tabs.forEach(tab => {
    tab.addEventListener('click', () => {
      tabs.forEach(t => t.classList.remove('active'));
      tab.classList.add('active');

      activeDay = tab×dataset×day;
      activeDateStr = tab×dataset×date ? formatDateForDisplay(tab.dataset.date) : '';

      filterItemsByDay(activeDay);
    });
  });

  // Filter items by day
  function filterItemsByDay(day) {
    const tbody = document×getElementById('invoice-items');
    const rows = tbody×querySelectorAll('tr');
    const emptyState = document.getElementById('empty-state');

    let visibleCount = 0;

    if (day === 'all') {
      // Show all rows and sort by date
      rows.forEach(row => {
        row.classList.remove('hidden');
        visibleCount++;
      });

      // Sort rows by date
      const rowsArray = Array×from(rows);
      rowsArray.sort((a, b) => {
        const dateA = parseDisplayDate(a×querySelector('.date-field').value);

```

```

        const dateB = parseDisplayDate(b×querySelector('.date-field').value);
        return dateA - dateB;
    });

    // Reappend in sorted order
    rowsArray.forEach(row => tbody.appendChild(row));
} else {
    // Get the date for this day
    const dayDate = dayMapping[day];
    const displayDate = formatDateForDisplay(dayDate);

    // Show only rows matching this date
    rows.forEach(row => {
        const dateField = row×querySelector('.date-field');
        if (dateField×value === displayDate) {
            row.classList.remove('hidden');
            visibleCount++;
        } else {
            row.classList.add('hidden');
        }
    });
}

// Show/hide empty state
if (visibleCount === 0) {
    emptyState.classList.remove('hidden');
    emptyState.classList.add('fade-in');
} else {
    emptyState.classList.add('hidden');
}

// Recalculate totals based on visible rows
calculateTotals();
}

// Parse display date (DD/MM/YY) to Date object
function parseDisplayDate(displayDate) {
    const parts = displayDate×split('/');
    if (parts×length !== 3) return new Date();

    const day = parseInt(parts[0]);

```

```

const month = parseInt(parts[1]) - 1;
const year = parseInt('20' + parts[2]);

return new Date(year, month, day);
}

// Set Monday-Friday
document.getElementById('setWeekdays').addEventListener('click', function() {
  const fromDate = document.getElementById('date-from');
  const toDate = document.getElementById('date-to');

  // Get the Monday of the current week
  const today = new Date();
  const day = today.getDay();
  const diff = today.getDate() - day + (day === 0 ? -6 : 1);
  const monday = new Date(today.setDate(diff));

  // Set Friday as 4 days after Monday
  const friday = new Date(monday);
  friday.setDate(monday.getDate() + 4);

  // Format dates for input
  fromDate.value = formatDateForInput(monday);
  toDate.value = formatDateForInput(friday);

  // Update day mapping
  updateDayMapping(monday);

  // Update fiscal week and invoice number
  updateFiscalWeekAndInvoiceNumber();
});

// Calculate fiscal week (UK tax year starts April 6)
function calculateFiscalWeek(date) {
  const currentYear = date.getFullYear();
  let fiscalYearStart;

  // If date is before April 6, fiscal year started previous year
  if (date.getMonth() < 3 || (date.getMonth() === 3 && date.getDate() < 6)) {
    fiscalYearStart = new Date(currentYear - 1, 3, 6); // April 6 previous year
  } else {

```

```

    fiscalYearStart = new Date(currentYear, 3, 6); // April 6 current year
  }

  // Calculate days since fiscal year start
  const daysSinceFiscalYearStart = Math.floor((date - fiscalYearStart) / (24 × 60 × 60
× 1000));

  // Calculate week number (add 1 because weeks are 1-indexed)
  const weekNumber = Math.floor(daysSinceFiscalYearStart / 7) + 1;

  return weekNumber;
}

// Update fiscal week display and invoice number
function updateFiscalWeekAndInvoiceNumber() {
  const fromDate = new Date(document.getElementById('date-from').value);
  const fiscalWeek = calculateFiscalWeek(fromDate);

  // Update fiscal week display
  document.getElementById('fiscal-week-display').textContent = `Week $
{fiscalWeek}`;

  // Generate invoice number: FY-WEEK-YEAR
  const year = fromDate.getFullYear();
  const month = fromDate.getMonth();
  let fiscalYear;

  // If date is before April 6, we're in the previous fiscal year
  if (month < 3 || (month === 3 && fromDate.getDate() < 6)) {
    fiscalYear = `${year-1}/${year.toString().slice(-2)}`;
  } else {
    fiscalYear = `${year}/${(year+1).toString().slice(-2)}`;
  }

  const invoiceNumber = `INV-${fiscalWeek}-${fiscalYear}`;
  document.getElementById('invoice-number').value = invoiceNumber;
}

// Listen for date changes to update fiscal week and invoice number
document.getElementById('date-from').addEventListener('change',
updateFiscalWeekAndInvoiceNumber);

```

```
document.getElementById('date-to').addEventListener('change',  
updateFiscalWeekAndInvoiceNumber);
```

```
// Update day mapping based on a Monday date
```

```
function updateDayMapping(mondayDate) {  
    const days = ['monday', 'tuesday', 'wednesday', 'thursday', 'friday'];  
    const startDate = new Date(mondayDate);  
  
    days.forEach((day, index) => {  
        const date = new Date(startDate);  
        date.setDate(startDate.getDate() + index);  
  
        // Update the mapping  
        dayMapping[day] = formatDateForInput(date);  
  
        // Update the tab data-date attribute  
        const tab = document.querySelector(`x-tab[data-day="${day}"]`);  
        if (tab) {  
            tab.dataset.date = formatDateForInput(date);  
        }  
    });  
}
```

```
// Format date for input field (YYYY-MM-DD)
```

```
function formatDateForInput(date) {  
    const year = date.getFullYear();  
    const month = String(date.getMonth() + 1).padStart(2, '0');  
    const day = String(date.getDate()).padStart(2, '0');  
    return `${year}-${month}-${day}`;  
}
```

```
// Format date for display (DD/MM/YY)
```

```
function formatDateForDisplay(dateStr) {  
    const date = new Date(dateStr);  
    const day = String(date.getDate()).padStart(2, '0');  
    const month = String(date.getMonth() + 1).padStart(2, '0');  
    const year = String(date.getFullYear()).slice(-2);  
    return `${day}/${month}/${year}`;  
}
```

```
// Add line item
```

```

document.getElementById('add-line').addEventListener('click', function() {
  addNewRow();
  calculateTotals();

  // Hide empty state if it was showing
  document.getElementById('empty-state').classList.add('hidden');
});

// Add all weekdays
document.getElementById('add-weekdays').addEventListener('click', function() {
  const fromDate = new Date(document.getElementById('date-from').value);
  const toDate = new Date(document.getElementById('date-to').value);

  // Clear existing rows
  const tbody = document.querySelector('invoice-items');
  tbody.innerHTML = '';

  // Add a row for each day between from and to dates
  for (let d = new Date(fromDate); d <= toDate; d.setDate(d.getDate() + 1)) {
    // Skip weekends
    if (d.getDay() !== 0 && d.getDay() !== 6) {
      const formattedDate = formatDateForDisplay(d.toISOString().split('T')[0]);
      addNewRow(formattedDate);
    }
  }

  // Update day mapping based on the from date (assuming it's a Monday)
  updateDayMapping(fromDate);

  // Filter to show the active day
  filterItemsByDay(activeDay);
});

// Add new row function
function addNewRow(dateValue = '') {
  const tbody = document.querySelector('invoice-items');
  const newRow = document.createElement('tr');
  newRow.className = 'table-row fade-in';

  if (!dateValue) {
    // If no date provided and a specific day is active, use that day's date
  }
}

```

```

if (activeDay !== 'all' && dayMapping[activeDay]) {
  dateValue = formatDateForDisplay(dayMapping[activeDay]);
} else {
  // Otherwise use the first date from the date picker
  const fromDateInput = document.getElementById('date-from');
  if (fromDateInput.value) {
    dateValue = formatDateForDisplay(fromDateInput.value);
  } else {
    dateValue = '';
  }
}
}

```

```

// Set the data-date attribute for filtering
newRow.dataset.date = dateValue;

```

```

newRow.innerHTML = `
  <td class="py-2 px-3 border-b border-gray-50">
    <input type="text" class="table-input text-sm date-field" value="$
{dateValue}">
  </td>
  <td class="py-2 px-3 border-b border-gray-50">
    <input type="text" class="table-input text-sm" placeholder="Location/Plot
Number">
  </td>
  <td class="py-2 px-3 border-b border-gray-50">
    <input type="text" class="table-input text-sm" placeholder="Description of
work performed">
  </td>
  <td class="py-2 px-3 border-b border-gray-50">
    <input type="number" class="table-input text-sm text-right hours-input"
value="" min="0" step="0.5" placeholder="">
  </td>
  <td class="py-2 px-3 border-b border-gray-50">
    <input type="text" class="table-input text-sm text-right rate-input" value=""
placeholder="£0.00">
  </td>
  <td class="py-2 px-3 border-b border-gray-50">
    <input type="text" class="table-input text-sm text-right amount-field"
value="£0.00" readonly>
  </td>

```

```

        <td class="py-2 px-3 border-b border-gray-50 text-center">
            <button class="text-gray-400 delete-row">x</button>
        </td>
    `;

```

```
tbody.appendChild(newRow);
```

```
// Add event listeners to the new row
```

```
setupRowEventListeners(newRow);
```

```
// If we're in a specific day view, hide the row if it doesn't match
```

```
if (activeDay !== 'all' && dayMapping[activeDay]) {
```

```
    const activeDate = formatDateForDisplay(dayMapping[activeDay]);
```

```
    if (dateValue !== activeDate) {
```

```
        newRow.classList.add('hidden');
```

```
    }
```

```
}
```

```
}
```

```
// Setup event listeners for a row
```

```
function setupRowEventListeners(row) {
```

```
    const dateField = row.querySelector('.date-field');
```

```
    const hoursInput = row.querySelector('.hours-input');
```

```
    const rateInput = row.querySelector('.rate-input');
```

```
    const deleteBtn = row.querySelector('.delete-row');
```

```
// Update row's data attribute when date changes
```

```
dateField.addEventListener('change', function() {
```

```
    row.dataset.date = dateField.value;
```

```
// If in day view, hide/show based on date
```

```
if (activeDay !== 'all') {
```

```
    const activeDate = formatDateForDisplay(dayMapping[activeDay]);
```

```
    if (dateField.value === activeDate) {
```

```
        row.classList.remove('hidden');
```

```
    } else {
```

```
        row.classList.add('hidden');
```

```
    }
```

```
}
```

```
});
```



```
hoursInput.addEventListener('input', function() {  
    calculateRowAmount(row);  
    calculateTotals();  
});
```

```
rateInput.addEventListener('input', function() {  
    // Format as currency  
    let value = rateInput.value.replace(/^[^0-9.]/g, '');  
    if (value) {  
        rateInput.value = '£' + parseFloat(value).toFixed(2);  
    } else {  
        rateInput.value = '';  
    }  
}
```

```
    calculateRowAmount(row);  
    calculateTotals();  
});
```

```
deleteBtn.addEventListener('click', function() {  
    // Add a subtle fade-out animation  
    row.style.transition = 'opacity 0.2s ease';  
    row.style.opacity = '0';
```

```
    setTimeout(() => {  
        row.remove();  
        calculateTotals();
```

```
        // Check if we need to show empty state  
        const visibleRows = document.querySelectorAll('#invoice-items  
tr:not(.hidden)');  
        if (visibleRows.length === 0) {  
            document.getElementById('empty-state').classList.remove('hidden');  
            document.getElementById('empty-state').classList.add('fade-in');  
        }  
    }, 200);  
});  
}
```

```
// Calculate row amount
```

```
function calculateRowAmount(row) {  
    const hoursInput = row.querySelector('.hours-input');
```

```
const rateInput = row.querySelector('.rate-input');
const amountField = row.querySelector('.amount-field');

const hours = parseFloat(hoursInput.value) || 0;
const rate = parseFloat(rateInput.value.replace(/^[^0-9.]/g, '')) || 0;
const amount = hours * rate;
```

```
amountField.value = '£' + amount.toFixed(2);
```

```
}
```

```
// Calculate totals
```

```
function calculateTotals() {
```

```
  const visibleRows = document.querySelectorAll('#invoice-items tr:not(.hidden)');
  let subtotal = 0;
```

```
  visibleRows.forEach(row => {
    const amountField = row.querySelector('.amount-field');
    subtotal += parseFloat(amountField.value.replace(/^[^0-9.]/g, '')) || 0;
  });
```

```
  const cisDeduction = subtotal * 0.2;
  const totalDue = subtotal - cisDeduction;
```

```
  document.getElementById('subtotal').textContent = '£' + subtotal.toFixed(2);
  document.getElementById('cis-deduction').textContent = '£' +
  cisDeduction.toFixed(2);
  document.getElementById('total-due').textContent = '£' + totalDue.toFixed(2);
}
```

```
// Save invoice with subtle animation
```

```
document.getElementById('save-invoice').addEventListener('click', function() {
```

```
  const button = this;
  button.disabled = true;
  button.textContent = 'Saving...';
  button.classList.add('opacity-75');
```

```
  setTimeout(() => {
    button.textContent = 'Saved!';
    button.classList.remove('opacity-75');
    button.classList.add('bg-green-500');
```

```

        setTimeout(() => {
            button.textContent = 'Save Invoice';
            button.classList.remove('bg-green-500');
            button.disabled = false;
        }, 1500);
    }, 800);
});

```

```

// Setup initial row event listeners
document.querySelectorAll('.table-row').forEach(row => {
    setupRowEventListeners(row);
});

```

```

// Update date mapping based on initial dates
const initialMonday = new Date(document.getElementById('date-from').value);
updateDayMapping(initialMonday);

```

```

// Initial calculation
calculateTotals();
});

```

```

</script>

```

```

<script>(function(){function c(){var b=a×contentDocument||
a.contentWindow.document;if(b){var
d=b.createElement('script');d.innerHTML="window.__CF$cv$params={r:'961a5c7c84b1ef37
',t:'MTc1MjkyOTYwMi4wMDAwMDA='};var
a=document×createElement('script');a×nonce='';a×src='/cdn-cgi/challenge-platform/
scripts/jsd/main.js';document.getElementsByTagName('head')
[0].appendChild(a);";b.getElementsByTagName('head')[0].appendChild(d)}}
if(document.body){var
a=document×createElement('iframe');a×height=1;a×width=1;a×style×position='absolute';a×
style×top=0;a×style×left=0;a×style×border='none';a×style×visibility='hidden';docu-
ment.body.appendChild(a);if('loading'!==document.readyState)c();else
if(window.addEventListener)document.addEventListener('DOMContentLoaded',c);else{var
e=document.onreadystatechange||function(){};document.onreadystatechange=function(b)
{e(b);'loading'!==document.readyState&&(document.onreadystatechange=e,c())}}})();</
script></body>
</html>

```